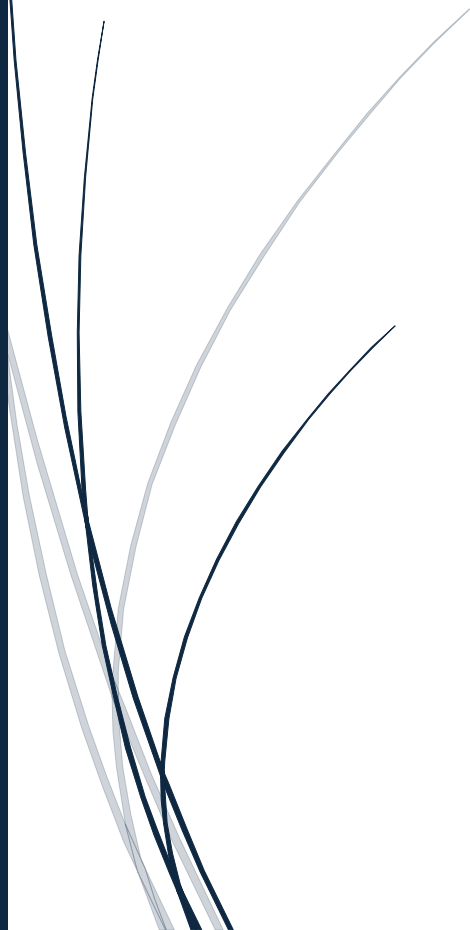




[Datum]

Developer Afspraken

[Ondertitel van document]



[BEDRIJFSNAAM]

Inhoud

Geen inhoudsopgavegegevens gevonden.

Inhoudsopgave

In dit onderstaande document vind u alle developer afspraken van het development team voor project [Project naam].

Deverloper Afspraken

Algemene afspraken

- We werken allemaal in Herd met een lokale Laravel-installatie.
- Iedere ontwikkelaar zorgt dat zijn/haar lokale omgeving functioneel en up-to-date is.
- De .env-bestanden worden nooit gedeeld via GitHub. Gevoelige data (zoals databasewachtwoorden of API-keys) blijven lokaal.
- Code wordt altijd getest en gereviewed voordat het gecommit wordt.

Projectstructuur

We volgen de standaard Laravel-structuur, met extra aandacht voor consistentie in Livewire:

Regels:

- Elke Livewire-component krijgt een eigen map met view (.blade.php) en class.
- Views worden logisch ingedeeld per pagina of functie.
- Blade-bestanden die hergebruikt worden, komen in resources/views/components.

Livewire richtlijnen

- Componentnamen zijn PascalCase in de PHP-class (bijv. UserProfile) en kebab-case in de Blade-view (bijv. <livewire:user-profile />).
- Data wordt zoveel mogelijk via public properties doorgegeven.
- Functies binnen Livewire-components zijn logisch gescheiden (geen database calls in render-methode).
- Gebruik emit() en listeners op een consistente manier voor communicatie tussen componenten.
- Gebruik @entangle() bij tweerichtingsbindingen (zoals inputs).

TailwindCSS afspraken

- We gebruiken Tailwind voor alle styling — geen inline CSS of extra CSS-bestanden, tenzij absoluut nodig.
- Klassen worden logisch geordend: eerst layout (flex, grid, spacing), daarna kleur/tekst.

- Herbruikbare UI-elementen (zoals knoppen, formulieren) worden gemaakt als Blade components of Livewire components.
- Houd design consistent — volg dezelfde spacing, kleuren en tekstgroottes.

Backend & Database

- Modelnamen zijn enkelvoud (bijv. User, Product).
- Tabellen in de database zijn meervoud (bijv. users, products).
- Relaties worden altijd gedefinieerd in het model (hasMany, belongsTo, etc.).
- Gebruik migrations en seeders voor databasebeheer — nooit handmatig tabellen aanmaken in phpMyAdmin.
- Alle seeders eindigen op zonder “Table” (bijv. UserSeeder i.p.v. UsersTableSeeder).

Git & Workflow

- Elke ontwikkelaar werkt in een eigen branch:
- Branches worden gemerged naar **main** via **pull requests** (niet direct).
- Commit messages zijn kort en duidelijk:
- Commit vaak en klein, niet alles in één commit.

Communicatie & Samenwerking

- Gebruik duidelijke commitberichten en beschrijvingen bij pull requests.
- Overleg bij grotere wijzigingen (zoals database-aanpassingen) eerst met het team.
- Elk teamlid houdt zijn werk gedocumenteerd (kort beschrijven wat er is toegevoegd of gewijzigd).

Extra afspraken

- Code is in het Engels (variabelen, functies, componentnamen).
- Commentaar mag in Nederlands als het de uitleg verduidelijkt.
- Houd het project schoon: verwijder oude, ongebruikte bestanden of code.
- Push niet naar main zonder dat de code getest en goedgekeurd is.
- Bij controles moeten 2 mensen het misnstens controleren

HANDTEKENINGEN

Ieder team lid moet hun handtekening hierin zetten.

Naam | Handtekening | Datum